

Neo Clouds — System Design

Status: Early access / working prototype
Site: neocloudsmarketplace.com
Version: 1.0 · September 2026
Source: [SYSTEM_DESIGN.md](#) · [Download PDF](#)

Control-plane architecture for an open marketplace that turns unused GPUs and TPUs into discoverable, reservable capacity. Supply aggregation and market discovery first; full orchestration, payments, and confidential multi-tenancy later.

Contents

1. [Goals and non-goals](#)

2. [Design principles](#)

3. [High-level architecture](#)

4. [Logical components](#)

5. [Domain model](#)

6. [Live agent protocol](#)

7. [Catalog and discovery](#)
8. [API map](#)

9. [Data and persistence](#)

10. [Trust model](#)

11. [Deployment](#)

12. [Roadmap](#)

13. [Buyer / seller flows](#)

14. [Summary](#)

1. Goals and non-goals

Goals

GOAL	DESIGN IMPLICATION
Unused → online	Providers list idle accelerators; live nodes require agent presence
Low / transparent cost	Listed \$/hr in catalog; payments disabled in early access
Open	Open APIs, open catalog axes, open-source control plane
Trust asymmetry reduced	Challenge attest (identity) ⇐/heartbeat (liveness)
Builder-friendly	REST + OpenAI-compatible inference surface

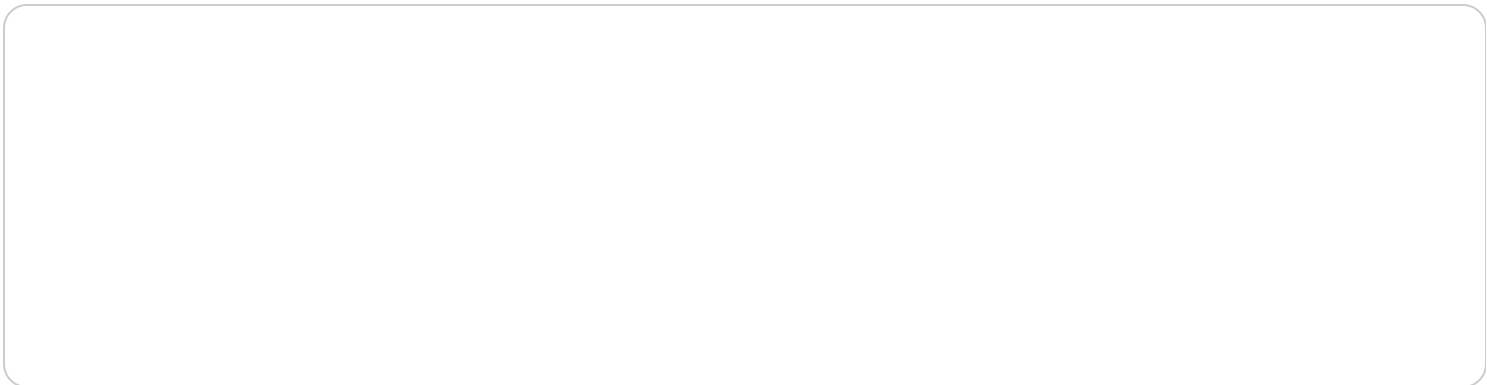
Non-goals (current generation)

- Automatic SSH user / container provisioning on provider hosts
- Payments, escrow, or payouts
- Cross-cloud job scheduler (Sky-style planner is a future consumer)
- Confidential computing / TEE-backed isolation
- Guaranteed SLA fleets at hyperscaler scale

2. Design principles

1. **Separate catalog truth from demo UX.** Seeded inventory may be simulated; live nodes are explicit and gated by attest + heartbeat.
2. **Thin provider integration.** Host-side `neo-agent` is enough to go live.
3. **Availability is derived.** Computed from attestation, heartbeat TTL, and reservation conflicts.
4. **Workload-aware discovery.** Hardware × workload tags × price.
5. **Honest early access.** UI/API advertise simulated vs live; no payment collection.

3. High-level architecture



Runtime: single Node.js process (`npm start`). **Host:** Render. **Domain:** neocloudsmarketplace.com via Cloudflare → Render.

4. Logical component view

```
flowchart LR
    subgraph Presentation
        Home["/ marketing"]
        Mkt["/marketplace"]
        Console["/console.html"]
        PilotPage["/providers.html"]
    end
    subgraph API["Marketplace API"]
        A["Auth"]
        N["Nodes"]
        L["Listings"]
        R["Reservations"]
        G["Agent"]
        I["Inference"]
    end
    subgraph Data
        S["store"]
        Q["SQLite"]
    end
    Home --> A
    Mkt --> L
    Mkt --> R
    Mkt --> I
    Console --> N
    Console --> L
    Console --> G
    A --> S
    N --> S
    L --> S
    R --> S
    G --> S
    I --> S
    S --> Q
```

MODULE	PATH	RESPONSIBILITY
Entry	src/marketplace.js	Boot, persistence, seed, listen
HTTP	src/server.js	Routes, static assets, health
Store	src/store.js	Process-local source of truth

MODULE	PATH	RESPONSIBILITY
DB	src/db.js	Debounced SQLite snapshot
Auth	src/auth.js	Accounts, API keys, roles
Providers	src/providers.js	Node register, stub attest, challenge begin
Agent	src/agent.js	Heartbeat, verify, pending, ack
Listings	src/listings.js	CRUD, filters, derived availability
Reservations	src/reservations.js	Simulated vs live lifecycle
Inference	src/inference.js	Models + chat completions

5. Core domain model

```
erDiagram
    ACCOUNT ||--o{ NODE : owns
    ACCOUNT ||--o{ LISTING : publishes
    ACCOUNT ||--o{ RESERVATION : buys
    NODE ||--o{ LISTING : backs
    LISTING ||--o{ RESERVATION : books
    NODE ||--o{ ATTEST_CHALLENGE : proves
    ACCOUNT {
        string account_id
        string role
        string api_key
    }
    NODE {
        string node_id
        boolean live
        string attestation_status
        datetime last_heartbeat_at
    }
    LISTING {
        string listing_id
        string price_per_hour
        boolean live
    }
    RESERVATION {
        string reservation_id
        string status
        boolean simulated
    }
```

Reservation status machine

```
stateDiagram-v2
    [*] --> active: demo / non-live reserve
    [*] --> pending_provision: live + online reserve
    pending_provision --> active: agent ack
    active --> completed: provider complete
    active --> cancelled: cancel
    pending_provision --> cancelled: cancel
```

KIND	ON RESERVE	CONNECTION INFO
Simulated / demo	<code>active</code> immediately	Placeholder note
Live (attested + heartbeat)	<code>pending_provision</code>	Filled on agent ack

6. Live hardware connect (agent protocol)

Aligned with the research paper: **attestation** $\neq$ **liveness**. Attest proves identity; heartbeat proves the node is online now.

```
sequenceDiagram
    participant P as Provider console
    participant API as Control plane
    participant A as neo-agent
    participant C as Customer
    P->>API: POST /v1/nodes live true
    API-->>P: node_id
    A->>API: attest challenge
    API-->>A: nonce
    A->>API: attest verify
    API-->>A: attested
    loop every 30s
        A->>API: heartbeat
        API-->>A: online + pending
    end
    P->>API: create listing
    C->>API: reserve
    API-->>C: pending_provision
    A->>API: pending + ack
    API-->>C: active + connection_info
```

Availability predicate (live)

1. Node exists and `attestation_status` $\equiv$ `attested`
2. Fresh heartbeat within TTL (default 90s)
3. No conflicting `active` / `pending_provision` reservation

METHOD	PATH	PURPOSE
POST	/v1/agent/heartbeat	Mark online
POST	/v1/nodes/:id/attest/challenge	Issue nonce
POST	/v1/agent/attest/verify	Bind fingerprint proof
GET	/v1/agent/nodes/:id/pending	List pending jobs
POST	/v1/agent/reservations/:id/ack	SSH / connection details

Host binary: `scripts/neo-agent.mjs` .

7. Catalog and discovery

AXIS	MECHANISM
Hardware	<code>accelerator_type</code> , <code>model</code> , <code>interconnect</code> , <code>min_vram_gb</code>
Workload	<code>workload</code> / <code>tags</code> (training, inference, fine-tune)
Price	<code>max_price_per_hour</code> , <code>sort=price_asc price_desc</code>
Availability	<code>available=true false</code>

8. API map

Public / lightly gated

Health, auth register/me, listings browse, models, stats, contact, provider pilot.

Provider-authenticated

Nodes, listings CRUD, challenge begin, agent routes, reservation complete.

Customer-authenticated

Reservations, chat completions, usage.

9. Data and persistence

```
flowchart LR
    Mutate["Domain mutation"] --> Store["store Maps"]
    Store --> Sched["schedulePersist"]
    Sched --> Snap["JSON snapshot"]
    Snap --> SQLite["SQLite kv"]
    Boot["initPersistence"] --> Load["loadSnapshot"]
    Load --> Store
```

- **Runtime source of truth:** in-process store
- **Durability:** SQLite via `NEO_DB_PATH`
- **Deploy caveat:** attach a persistent disk for `data/` on Render

10. Trust model

RISK	CURRENT MITIGATION	FUTURE
Stale availability	Heartbeat TTL	SLA / reputation
Fake hardware	Challenge + fingerprint; stub blocked on live	Device attestation
Malicious workloads	Provider-managed SSH	Containers / VMs / TEEs
Key leakage	Bearer API keys	Scoped keys, rotation

Payments: disabled. Listed prices are discovery signals only.

11. Deployment

```
flowchart TB
    User["Client"] --> DNS["DNS"]
    DNS --> CF["Cloudflare"]
    CF --> Render["Render web service"]
    Render --> App["marketplace.js"]
    App --> Disk["SQLite"]
    AgentHost["Provider GPU host"] --> CF
```

12. Target architecture (roadmap)

```
flowchart TB
    subgraph Now["Now v1"]
        CP["API + agent"]
        Cat["Catalog H x W x P"]
        SQL["SQLite"]
    end
    subgraph Next["Next"]
        Prov["Auto provision"]
        Pay["Payments"]
        PG["Postgres"]
        InfLive["Live inference"]
    end
    subgraph Later["Later"]
        Match["Matching engine"]
        Secure["TEE tenancy"]
        Sky["Sky planners"]
        Trace["Public traces"]
    end
    Now --> Next --> Later
```

13. Key sequences

Seller (live)

1. Provider key → `/console.html`
2. Register live node
3. Run `neo-agent`
4. Publish listing
5. Ack customer reservations

Buyer

1. Browse `/marketplace`
2. Get customer API key
3. Reserve hours
4. LIVE → wait for `active` + `connection_info`
5. SIMULATED → demo reservation only

14. Summary

Neo Clouds is a **marketplace control plane**: auth, node identity, listing discovery, reservation lifecycle, and an agent protocol that binds live supply to attested, heartbeating hosts. It ships an open unused-accelerator exchange first, then grows toward matching, secure multi-tenancy, and measured utilization.